Identificação do projeto

Título do projeto: PJBL Steam Analytics — preço, gênero, reviews e pico de jogadores simultâneos na Steam
Integrantes do grupo: Lucas Stopinski da Silva (RA 40041246) • Lucas Bruno e Silva (RA 40090552)
Turma: Data Analytics (PjBL) • PUCPR • Turma U • 2026/2
Descrição breve do problema ou contexto: A Steam publica, para cada jogo, preço, gêneros, tags, reviews e uma estimativa de pico de jogadores simultâneos (Peak CCU). O projeto investiga quais características de um jogo acompanham maior audiência e como o catálogo histórico se compara ao comportamento dos jogos mais jogados em tempo real.

Notebook único do projeto, preenchido ao longo de RA1 e RA2. O detalhamento completo do RA1 está em `Notebooks/01_inspect_prepare.ipynb` e em `docs/RA1_Data_Analytics.docx`; aqui o fluxo foi condensado e é reexecutável do início ao fim.

RA1 — Data Ingestion, Cleaning and Initial Analysis

Nesta etapa, carregue, prepare e explore os dados do projeto.

1. Contexto e objetivo

Apresente brevemente o problema investigado e o que o grupo pretende descobrir com os dados.

Resposta do grupo:

A Steam é a maior loja de jogos para PC e tem um catálogo grande (mais de 125 mil títulos), heterogêneo e com muitos jogos sem telemetria de audiência. Desenvolvedoras e analistas querem saber **quais características de um jogo (preço, gênero/tags, nota de review) estão associadas a um pico maior de jogadores simultâneos (`Peak CCU`)** e como o catálogo histórico se compara ao comportamento ao vivo dos jogos mais jogados.

Perguntas analíticas do projeto: P1 (preço × `Peak CCU`), P2 (gêneros e tags com maior `Peak CCU`), P3 (nota de review e Metacritic × `Peak CCU`) e P4 (histórico × dado ao vivo). No RA1 o objetivo é apenas obter, entender e preparar o catálogo; as respostas ficam para as próximas etapas.

2. Fonte e ingestão dos dados

- Identifique a fonte dos dados, incluindo link ou referência de acesso.
- Descreva brevemente o dataset e o que seus registros representam.
- Explique como os dados foram carregados e implemente a ingestão.

Resposta do grupo:

Fonte principal: fronkongames/steam-games-dataset (Kaggle, acesso em 07/10/2026), arquivo `games.csv`, 401 MB, CSV UTF-8, **125.855 jogos × 40 campos**. Cada registro representa **um jogo ou aplicativo da Steam**, identificado por `AppID`.

Ingestão: o arquivo não vai para o Git (limite de 100 MB). Para obtê-lo, baixe `games.csv` da pasta do Drive do grupo (`Dataset/fronkongames/`) e salve em `Dataset/fronkongames/games.csv`. Alternativa pelo Kaggle (exige `pip install -r requirements.txt` e token em `~/.kaggle/kaggle.json`): `kaggle datasets download -d fronkongames/steam-games-dataset -p Dataset/fronkongames --unzip`. A célula abaixo valida a presença do arquivo e faz a leitura.

In [1]:
```
import csv
from pathlib import Path

import pandas as pd

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)

CAMINHO_ORIGINAL = Path("../Dataset/fronkongames/games.csv")
assert CAMINHO_ORIGINAL.exists(), (
    f"{CAMINHO_ORIGINAL} não encontrado. Baixe com o comando kaggle indicado acima."
)
print(f"Base original encontrada: {CAMINHO_ORIGINAL} ({CAMINHO_ORIGINAL.stat().st_size / 1e6:.0f} MB)")

# Leitura ingênua: serve de diagnóstico, o resultado sai desalinhado (ver seção 3).
df_bruto = pd.read_csv(CAMINHO_ORIGINAL)
print(f"Leitura ingênua: {df_bruto.shape[0]} linhas x {df_bruto.shape[1]} colunas")
```

```
Base original encontrada: ../Dataset/fronkongames/games.csv (401 MB)
Leitura ingênua: 125855 linhas x 39 colunas
```

3. Exploração inicial dos dados

Investigue a quantidade de registros e colunas, os tipos de dados, os valores ausentes e outras características relevantes da base.

In [2]:
```
# Cabeçalho x primeira linha de dados, via csv.reader (aspas tratadas
corretamente)
with open(CAMINHO_ORIGINAL, newline="", encoding="utf-8") as f:
    leitor = csv.reader(f)
    cabecalho = next(leitor)
    primeira_linha = next(leitor)

print(f"Campos no cabeçalho: {len(cabecalho)} | campos na 1ª linha de dado:
{len(primeira_linha)}")
print("Trecho do cabeçalho ao redor do ponto suspeito:", cabecalho[6:10])
print("Amostra de 'Release date' na leitura ingênua (deveria ser data):")
print(df_bruto["Release date"].sample(5, random_state=2).tolist())
print(f"AppID nulo na leitura ingênua: {df_bruto['AppID'].isnull().sum()}")
```

```
Campos no cabeçalho: 39 | campos na 1ª linha de dado: 40
Trecho do cabeçalho ao redor do ponto suspeito: ['Price', 'DiscountDLC count',
'About the game', 'Supported languages']
Amostra de 'Release date' na leitura ingênua (deveria ser data):
['200000 - 500000', '0 - 0', '0 - 20000', '0 - 20000', '0 - 20000']
AppID nulo na leitura ingênua: 1
```

In [3]:
```
# Releitura com cabeçalho corrigido: "DiscountDLC count" era a fusão de
"Discount" + "DLC count"
cabecalho_corrigido = cabecalho[:8] + ["Discount"] + cabecalho[8:]
df = pd.read_csv(CAMINHO_ORIGINAL, skiprows=1, header=None,
names=cabecalho_corrigido)
print(f"Leitura corrigida: {df.shape[0]} linhas x {df.shape[1]} colunas |
AppID nulo: {df['AppID'].isnull().sum()}")

total = len(df)
resumo = pd.DataFrame({
    "verificação": [
        "Coluna 100% vazia (Movies)",
        "Metacritic score == 0 (sentinela de 'sem nota')",
        "Genres ausentes", "Tags ausentes",
        "Linhas duplicadas (exatas)", "AppID duplicado", "Name repetido
(AppID distinto)",
        "Peak CCU == 0 (sem telemetria)",
    ],
    "resultado": [
        f"{df['Movies'].isna().sum()} de {total} nulos",
        f"{(df['Metacritic score'] == 0).sum()} ({(df['Metacritic score'] ==
0).mean():.1%})",
        f"{df['Genres'].isna().sum()} ({df['Genres'].isna().mean():.1%})",
        f"{df['Tags'].isna().sum()} ({df['Tags'].isna().mean():.1%})",
        int(df.duplicated().sum()), int(df["AppID"].duplicated().sum()),

int(df["Name"].astype(str).str.strip().str.lower().duplicated().sum()),
        f"{(df['Peak CCU'] == 0).sum()} ({(df['Peak CCU'] ==
0).mean():.1%})",
    ],
})
display(resumo.style.hide(axis="index"))
```

```
print("Estimated owners (texto em faixa):", df["Estimated
owners"].head(3).tolist())
```

```
Leitura corrigida: 125855 linhas x 40 colunas | AppID nulo: 0
```

verificação	resultado
Coluna 100% vazia (Movies)	125855 de 125855 nulos
Metacritic score == 0 (sentinela de 'sem nota')	121597 (96.6%)
Genres ausentes	8423 (6.7%)
Tags ausentes	42502 (33.8%)
Linhas duplicadas (exatas)	0
AppID duplicado	0
Name repetido (AppID distinto)	1593
Peak CCU == 0 (sem telemetria)	106175 (84.4%)

```
Estimated owners (texto em faixa): ['0 – 0', '0 – 20000', '0 – 20000']
```

Resposta do grupo:

- **Cabeçalho malformado (problema mais grave):** 39 nomes contra 40 campos por linha; `"DiscountDLC count"` é a fusão de `Discount` e `DLC count`. Na leitura ingênua quase todas as colunas ficam desalinhadas (`Release date` vem com faixas de owners, `AppID` tem 1 nulo).
- `Movies` está 100% vazia; `Estimated owners` é texto em faixa (`"0 – 20000"`); `Metacritic score` usa 0 como "sem nota" em 96,6% das linhas.
- Ausências em `Genres` , `Developers` , `Categories` , `Publishers` e `Tags` (de 6,7% a 33,8%).
- Sem duplicatas de linha ou de `AppID` ; 1.593 nomes repetidos com `AppID` distinto (demos, remasters).
- `Peak CCU` é 0 em 84,4% dos jogos: é **ausência de telemetria**, não pico zero. Valores extremos de `Price` e `Peak CCU` são reais (ex.: Counter-Strike 2 com 1.013.936).

Verificações completas em `Notebooks/01_inspect_prepare.ipynb` .

4. Limpeza e preparação dos dados

Realize a limpeza necessária ao projeto. Verifique valores ausentes, duplicados e inconsistências e trate os problemas encontrados, quando necessário.

In [4]:
```
df_limpo = df.copy(deep=True)

# 1) Coluna Movies 100% vazia: sem informação, removida
df_limpo = df_limpo.drop(columns=["Movies"])
```

```python
# 2) Metacritic score: 0 significa "sem nota", não nota mínima
df_limpo["Metacritic score"] = df_limpo["Metacritic score"].replace(0,
pd.NA)

# 3) Ausências em categóricas: marcador explícito mantém o jogo nos
agrupamentos
categoricas = ["Genres", "Developers", "Categories", "Publishers", "Tags"]
df_limpo[categoricas] = df_limpo[categoricas].fillna("Not specified")

# 4) Duplicatas exatas: verificação (a base não tem; AppID é a chave, Name
pode repetir)
df_limpo = df_limpo.drop_duplicates().reset_index(drop=True)

# 5) Outliers de Price/Peak CCU/Required age: mantidos, são valores reais;
só se checa o impossível
assert (df_limpo[["Price", "Peak CCU"]] >= 0).all().all() and
df_limpo["Required age"].between(0, 21).all()

print(f"Após a limpeza: {df_limpo.shape[0]} linhas x {df_limpo.shape[1]}
colunas")
print(f"Metacritic == 0: {(df_limpo['Metacritic score'] == 0).sum()} | NaN:
{df_limpo['Metacritic score'].isna().sum()}")
print(f"Ausentes em categóricas:
{int(df_limpo[categoricas].isna().sum().sum())}")
```

```
Após a limpeza: 125855 linhas x 39 colunas
Metacritic == 0: 0 | NaN: 121597
Ausentes em categóricas: 0
```

Resposta do grupo:

Problema	Tratamento	Efeito
Cabeçalho com 39 nomes para 40 campos	Cabeçalho reconstruído (inserido `Discount`)	125.855 linhas realinhadas, `AppID` sem nulos
`Movies` 100% vazia	Coluna removida	-1 coluna
`Metacritic score` == 0 em 121.597 linhas	0 convertido em NaN	médias passam a refletir só as 4.258 notas reais
Ausências em 5 colunas categóricas	Preenchidas com `"Not specified"`	nenhuma linha perdida em agrupamentos
Duplicatas exatas	Verificadas: 0	nada removido
Nomes repetidos (1.593)	Mantidos	`AppID` é a chave real
Outliers de `Price`, `Peak CCU`, `Required age`	Mantidos	são valores reais; `Peak CCU` == 0 não foi imputado

5. Transformação dos dados

Realize as transformações necessárias à análise, como criação de colunas, filtros, agrupamentos, conversões ou junções, quando aplicáveis.

In [5]:
```
# Estimated owners: faixa em texto -> colunas numéricas
faixa = df_limpo["Estimated owners"].str.split(" - ", expand=True)
df_limpo["owners_min"] = pd.to_numeric(faixa[0], errors="coerce")
df_limpo["owners_max"] = pd.to_numeric(faixa[1], errors="coerce")
df_limpo["owners_avg"] = (df_limpo["owners_min"] + df_limpo["owners_max"]) /
2

# Release date: texto -> datetime
df_limpo["Release date"] = pd.to_datetime(df_limpo["Release date"],
errors="coerce", format="mixed")

print(f"owners_* ausentes:
{int(df_limpo[['owners_min','owners_max','owners_avg']].isna().sum().sum())}
")
print(f"Release date: {df_limpo['Release date'].dtype} | datas não
convertidas: {df_limpo['Release date'].isna().sum()}")
```

```
owners_* ausentes: 0
Release date: datetime64[us] | datas não convertidas: 0
```

Resposta do grupo:

`Estimated owners` virou três colunas numéricas (`owners_min`, `owners_max`, `owners_avg`, esta última o ponto médio da faixa) e `Release date` virou `datetime`, o que permite análises numéricas e temporais nas próximas etapas. A coluna original de owners foi mantida. Nenhuma agregação analítica foi feita neste RA.

6. Validação do conjunto preparado

Confirme que o conjunto final tem o formato esperado antes de seguir para as próximas etapas.

In [6]:
```
assert df_limpo.shape == (125855, 42), df_limpo.shape
assert df_limpo["AppID"].is_unique and df_limpo["AppID"].notna().all()
assert "Movies" not in df_limpo.columns
print(f"Conjunto preparado: {df_limpo.shape[0]} linhas x {df_limpo.shape[1]}
colunas | AppID único")
```

```
Conjunto preparado: 125855 linhas x 42 colunas | AppID único
```

Resposta do grupo:

O conjunto preparado tem 125.855 linhas e 42 colunas, `AppID` único e sem a coluna `Movies`, como esperado.

7. Considerações do RA1

Registre brevemente:

- O que o grupo descobriu sobre os dados.
- Quais dificuldades ou limitações encontrou.
- Como os dados preparados poderão ser utilizados nas próximas etapas.

Resposta do grupo:

- **O que foi descoberto:** o dataset tinha um defeito estrutural (cabeçalho) que invalidaria qualquer análise; corrigido, a base fica íntegra (125.855 × 42, `AppID` único). 84,4% dos jogos não têm telemetria de `Peak CCU`.
- **Limitações:** as perguntas P1 a P3 valem só para o subconjunto com `Peak CCU` > 0 (cerca de 16%); Metacritic existe em 3,4% dos jogos; `owners_avg` é ponto médio de faixa.
- **Próximas etapas:** `df_limpo` é o ponto de partida do RA2. Como `Peak CCU` é a variável mais confiável da base, ela serve de referência para simular, no RA2, a chegada contínua de leituras de jogadores dos jogos mais jogados.

RA2 — Streaming Data Processing and Time Windows

Utilize os dados e o contexto do projeto para demonstrar o processamento em streaming e a aplicação de janelas temporais. Escolha um dos cenários:

- **Cenário A — Dados estáticos:** simule a chegada de novos registros utilizando dados artificiais coerentes com o contexto do projeto.
- **Cenário B — Dados em streaming:** utilize sua própria fonte de dados contínuos, sem necessidade de simulação.

8. Objetivo da análise em streaming

Responda brevemente:

- O que o grupo deseja monitorar ou analisar?
- Qual informação espera obter com o processamento em streaming?

Resposta do grupo:

Queremos **monitorar, em janelas de 5 minutos, a média de jogadores simultâneos dos jogos mais jogados da Steam** enquanto novas leituras chegam. A informação esperada é como o nível de audiência de cada jogo evolui ao longo do tempo (tendência de subida ou queda), sem esperar o fim da coleta, que é a ideia do processamento em streaming. Isso responde, de forma simplificada, à pergunta P4 do projeto (histórico x comportamento ao vivo).

9. Fonte de dados e processamento

- Informe o cenário escolhido: dados reais em streaming ou simulação.
- Explique brevemente como os novos registros chegam.
- Implemente o processamento em tempo real.
- Demonstre o recebimento de novos registros.

Resposta do grupo:

Cenário A — dados estáticos com simulação. A base do projeto é um catálogo fixo; por isso as leituras que chegam em streaming são **artificiais, mas coerentes com a base**: pegamos os 3 jogos de maior `Peak CCU` de `df_limpo` e, para cada um, geramos uma leitura de jogadores simultâneos por minuto durante 15 minutos (45 eventos). O valor de cada leitura parte de uma fração do `Peak CCU` real do jogo, com tendência de alta e ruído pequeno, e nunca ultrapassa esse pico.

As leituras chegam em **3 lotes**, como arquivos JSON numa pasta monitorada pelo Spark Structured Streaming (`readStream` com schema explícito). Cada lote é escrito numa pasta de publicação e só então movido para a pasta de entrada, para o Spark nunca ler arquivo incompleto.

In [7]:
```
import json
import random
import tempfile
from pathlib import Path
from uuid import uuid4

from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, round as arredonda, window
from pyspark.sql.types import IntegerType, StringType, StructField,
StructType, TimestampType

spark = (
    SparkSession.builder
    .master("local[2]")
    .appName("Steam streaming — RA2")
    .config("spark.sql.session.timeZone", "America/Sao_Paulo")
    .config("spark.sql.shuffle.partitions", "2")
    .getOrCreate()
)
```

```
spark.sparkContext.setLogLevel("ERROR")
print("Spark", spark.version, "| fuso da sessão:",
spark.conf.get("spark.sql.session.timeZone"))

# Os 3 jogos de maior Peak CCU da base preparada no RA1
top3 = df_limpo.nlargest(3, "Peak CCU")[["AppID", "Name", "Peak
CCU"]].reset_index(drop=True)
display(top3)
```

```
Spark 4.0.3 | fuso da sessão: America/Sao_Paulo
```

	AppID	Name	Peak CCU
0	730	Counter-Strike 2	1013936
1	570	Dota 2	623941
2	578080	PUBG: BATTLEGROUNDS	314682

In [8]:
```
# Geração das 45 leituras (3 jogos x 15 minutos). Determinística: mesma
semente, mesmos dados.
rng = random.Random(42)
INICIO = pd.Timestamp("2026-10-21 14:00:00")
MINUTOS = 15

leituras = []
for minuto in range(MINUTOS):
    for jogo in top3.to_dict("records"):
        fracao = min(0.70 + 0.02 * minuto + rng.uniform(-0.03, 0.03), 1.0)
# tendência de alta + ruído
        leituras.append({
            "timestamp": (INICIO +
pd.Timedelta(minutes=minuto)).strftime("%Y-%m-%d %H:%M:%S"),
            "appid": int(jogo["AppID"]),
            "jogo": jogo["Name"],
            "jogadores": int(jogo["Peak CCU"] * fracao),
        })

eventos = pd.DataFrame(leituras)
picos = top3.set_index("AppID")["Peak CCU"]
assert (eventos.groupby("appid")["jogadores"].max() <=
picos.reindex(sorted(picos.index))).all()
print(f"{len(eventos)} leituras geradas. Primeiras linhas:")
display(eventos.head(6))
```

```
45 leituras geradas. Primeiras linhas:
```

	timestamp	appid	jogo	jogadores
0	2026-10-21 14:00:00	730	Counter-Strike 2	718237
1	2026-10-21 14:00:00	570	Dota 2	418976
2	2026-10-21 14:00:00	578080	PUBG: BATTLEGROUNDS	216029
3	2026-10-21 14:01:00	730	Counter-Strike 2	713195
4	2026-10-21 14:01:00	570	Dota 2	458090
5	2026-10-21 14:01:00	578080	PUBG: BATTLEGROUNDS	229907

In [9]:
```
# Estrutura de pastas e fonte de streaming (padrão visto em aula: schema explícito + pasta monitorada)
area_temporaria = tempfile.TemporaryDirectory(prefix="steam_stream_")
base = Path(area_temporaria.name)
pasta_entrada, pasta_publicacao, pasta_checkpoint = base / "entrada", base / "publicacao", base / "checkpoint"
pasta_entrada.mkdir(); pasta_publicacao.mkdir()

schema = StructType([
    StructField("timestamp", TimestampType(), True),
    StructField("appid", IntegerType(), True),
    StructField("jogo", StringType(), True),
    StructField("jogadores", IntegerType(), True),
])

leituras_stream = (
    spark.readStream
    .schema(schema)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .json(str(pasta_entrada))
)
print("É um DataFrame de streaming?", leituras_stream.isStreaming)
print("Arquivos na pasta de entrada:", list(pasta_entrada.iterdir()))
```

```
É um DataFrame de streaming? True
Arquivos na pasta de entrada: []
```

10. Aplicação de janelas temporais

- Utilize uma janela **Tumbling** (intervalos sem sobreposição) ou **Sliding** (intervalos que podem se sobrepor).
- Informe o tipo e a duração da janela e o deslocamento, quando aplicável.
- Implemente uma agregação coerente com a análise, como contagem, soma ou média.
- Explique brevemente o que será calculado em cada intervalo.

O foco é demonstrar a compreensão do conceito; não é necessário justificar tecnicamente a duração escolhida.

Resposta do grupo:

- **Tipo:** janela **Tumbling** (intervalos consecutivos, sem sobreposição).
- **Duração:** 5 minutos. **Deslocamento:** não se aplica (na tumbling o deslocamento é igual à duração).
- **Agregação: média** de `jogadores` (`avg`), calculada por janela **e por jogo**.
- **O que cada janela calcula:** a média de jogadores simultâneos de cada jogo nas leituras cujo `timestamp` cai naquele intervalo de 5 minutos. Início incluído, fim excluído: a leitura das 14:05:00 entra na janela 14:05-14:10. Cada leitura pertence a uma única janela.

In [10]:
```
medias_por_janela = (
    leituras_stream
    .groupBy(window("timestamp", "5 minutes"), "jogo")
    .agg(arredonda(avg("jogadores"),
0).cast("long").alias("media_jogadores"))
)

nome_tabela = "medias_" + uuid4().hex
consulta = (
    medias_por_janela.writeStream
    .format("memory")
    .queryName(nome_tabela)
    .outputMode("complete")          # a tabela de resultado é reescrita
inteira a cada lote
    .option("checkpointLocation", str(pasta_checkpoint))
    .start()
)
print("Consulta ativa:", consulta.isActive)

def resultado():
    return (
        spark.table(nome_tabela)
        .select(
            "window.start", "window.end", "jogo", "media_jogadores",
        )
        .orderBy("start", "jogo")
        .toPandas()
    )

def chegada_de_lote(numero, inicio_min, fim_min):
    lote = eventos.iloc[inicio_min * 3:(fim_min + 1) * 3]            # 3
jogos por minuto
    arquivo = pasta_publicacao / f"lote_{numero}_{uuid4().hex}.json"
    with arquivo.open("w", encoding="utf-8") as f:
        for linha in lote.to_dict("records"):
            f.write(json.dumps(linha) + "\n")
    arquivo.replace(pasta_entrada / arquivo.name)                     #
publica só com o arquivo já fechado
    consulta.processAllAvailable()
    print(f"Lote {numero}: {len(lote)} leituras (minutos {inicio_min} a
{fim_min}) | "
          f"última leitura recebida: {lote['timestamp'].iloc[-1]}")
    return lote
```

```
Consulta ativa: True
```

Cada lote abaixo simula uma chegada de dados em um momento diferente. Após cada um, mostramos a tabela de resultados: as janelas já completas permanecem iguais e a janela em curso é atualizada.

11. Resultados e interpretação

Apresente os resultados do processamento e demonstre como a análise se comporta com a chegada de novos registros.

In [11]:
```
# Lote 1 — minutos 0 a 6: a janela 14:00–14:05 fica completa e a 14:05–14:10 recebe só 2 minutos
chegada_de_lote(1, 0, 6)
res1 = resultado()
display(res1)
```

```
Lote 1: 21 leituras (minutos 0 a 6) | última leitura recebida: 2026-10-21 14:06:00
```

	start	end	jogo	media_jogadores
0	2026-10-21 14:00:00	2026-10-21 14:05:00	Counter-Strike 2	741931
1	2026-10-21 14:00:00	2026-10-21 14:05:00	Dota 2	452476
2	2026-10-21 14:00:00	2026-10-21 14:05:00	PUBG: BATTLEGROUNDS	232973
3	2026-10-21 14:05:00	2026-10-21 14:10:00	Counter-Strike 2	832067
4	2026-10-21 14:05:00	2026-10-21 14:10:00	Dota 2	490922
5	2026-10-21 14:05:00	2026-10-21 14:10:00	PUBG: BATTLEGROUNDS	258622

In [12]:
```
# Lote 2 — minutos 7 a 11: completa a janela 14:05–14:10 e abre a 14:10–14:15
chegada_de_lote(2, 7, 11)
res2 = resultado()
display(res2)
```

```
Lote 2: 15 leituras (minutos 7 a 11) | última leitura recebida: 2026-10-21 14:11:00
```

	start	end	jogo	media_jogadores
0	2026-10-21 14:00:00	2026-10-21 14:05:00	Counter-Strike 2	741931
1	2026-10-21 14:00:00	2026-10-21 14:05:00	Dota 2	452476
2	2026-10-21 14:00:00	2026-10-21 14:05:00	PUBG: BATTLEGROUNDS	232973
3	2026-10-21 14:05:00	2026-10-21 14:10:00	Counter-Strike 2	859085
4	2026-10-21 14:05:00	2026-10-21 14:10:00	Dota 2	518504
5	2026-10-21 14:05:00	2026-10-21 14:10:00	PUBG: BATTLEGROUNDS	263377
6	2026-10-21 14:10:00	2026-10-21 14:15:00	Counter-Strike 2	946415
7	2026-10-21 14:10:00	2026-10-21 14:15:00	Dota 2	569813
8	2026-10-21 14:10:00	2026-10-21 14:15:00	PUBG: BATTLEGROUNDS	287194

In [13]:
```
# Lote 3 — minutos 12 a 14: completa a janela 14:10–14:15
chegada_de_lote(3, 12, 14)
res3 = resultado()
display(res3)
```

```
Lote 3: 9 leituras (minutos 12 a 14) | última leitura recebida: 2026-10-21
14:14:00
```

	start	end	jogo	media_jogadores
0	2026-10-21 14:00:00	2026-10-21 14:05:00	Counter-Strike 2	741931
1	2026-10-21 14:00:00	2026-10-21 14:05:00	Dota 2	452476
2	2026-10-21 14:00:00	2026-10-21 14:05:00	PUBG: BATTLEGROUNDS	232973
3	2026-10-21 14:05:00	2026-10-21 14:10:00	Counter-Strike 2	859085
4	2026-10-21 14:05:00	2026-10-21 14:10:00	Dota 2	518504
5	2026-10-21 14:05:00	2026-10-21 14:10:00	PUBG: BATTLEGROUNDS	263377
6	2026-10-21 14:10:00	2026-10-21 14:15:00	Counter-Strike 2	964231
7	2026-10-21 14:10:00	2026-10-21 14:15:00	Dota 2	588157
8	2026-10-21 14:10:00	2026-10-21 14:15:00	PUBG: BATTLEGROUNDS	294198

In [14]:
```
# Comparação lado a lado: como cada janela mudou conforme os lotes chegaram
comparacao = (
```

```
    res1.rename(columns={"media_jogadores": "após lote 1"})
    .merge(res2.rename(columns={"media_jogadores": "após lote 2"}), on=
["start", "end", "jogo"], how="outer")
    .merge(res3.rename(columns={"media_jogadores": "após lote 3"}), on=
["start", "end", "jogo"], how="outer")
    .sort_values(["start", "jogo"])
)
comparacao[["após lote 1", "após lote 2", "após lote 3"]] =
comparacao[["após lote 1", "após lote 2", "após lote 3"]].astype("Int64")
comparacao["start"] = comparacao["start"].dt.strftime("%H:%M")
comparacao["end"] = comparacao["end"].dt.strftime("%H:%M")
display(comparacao.set_index(["start", "end", "jogo"]))

# Verificação: o resultado do streaming deve ser igual ao cálculo em batch
sobre os mesmos 45 eventos
batch = (
    eventos.assign(timestamp=pd.to_datetime(eventos["timestamp"]))
    .groupby([pd.Grouper(key="timestamp", freq="5min"), "jogo"])
["jogadores"].mean().round()
    .rename("media_jogadores").reset_index().rename(columns={"timestamp":
"start"})
    .sort_values(["start", "jogo"]).reset_index(drop=True)
)
final = res3[["start", "jogo", "media_jogadores"]].sort_values(["start",
"jogo"]).reset_index(drop=True)
assert (batch["media_jogadores"].astype("int64") ==
final["media_jogadores"].astype("int64")).all()
assert (batch["jogo"] == final["jogo"]).all()
print("Streaming e batch coincidem nas 9 médias (3 janelas x 3 jogos).")
```

			após lote 1	após lote 2	após lote 3
start	**end**	**jogo**			
14:00	**14:05**	**Counter-Strike 2**	741931	741931	741931
		Dota 2	452476	452476	452476
		PUBG: BATTLEGROUNDS	232973	232973	232973
14:05	**14:10**	**Counter-Strike 2**	832067	859085	859085
		Dota 2	490922	518504	518504
		PUBG: BATTLEGROUNDS	258622	263377	263377
14:10	**14:15**	**Counter-Strike 2**	<NA>	946415	964231
		Dota 2	<NA>	569813	588157
		PUBG: BATTLEGROUNDS	<NA>	287194	294198

```
Streaming e batch coincidem nas 9 médias (3 janelas x 3 jogos).
```

Resposta do grupo:

- **Janela 14:00-14:05 (completa no lote 1):** não muda mais depois disso (Counter-Strike 2: 741.931; Dota 2: 452.476; PUBG: 232.973). Uma janela fechada só mudaria

se chegassem leituras atrasadas para ela.

- **Janela 14:05-14:10 (em curso):** após o lote 1 só tinha os minutos 5 e 6, e a média do Counter-Strike 2 era 832.067; com os minutos 7 a 9 do lote 2 passou a 859.085. Dota 2 foi de 490.922 para 518.504 e PUBG de 258.622 para 263.377. É a mudança que mostra o streaming funcionando: **o resultado é recalculado quando novos registros chegam**, sem reprocessar tudo.
- **Janela 14:10-14:15:** só existe a partir do lote 2 (minutos 10 e 11) e é ajustada no lote 3 (Counter-Strike 2: 946.415 para 964.231; Dota 2: 569.813 para 588.157; PUBG: 287.194 para 294.198).
- **Tendência:** as médias sobem de janela em janela nos três jogos, coerente com a tendência de alta usada na simulação (e sempre abaixo do `Peak CCU` real de cada jogo).
- **Conferência:** o resultado final do streaming é idêntico ao cálculo em batch sobre as mesmas 45 leituras (verificação com `assert`), o que valida o processamento.

In [15]:
```
# Encerramento: para a consulta e remove a tabela e as pastas temporárias
consulta.stop()
spark.catalog.dropTempView(nome_tabela)
area_temporaria.cleanup()
print("Consulta ativa:", consulta.isActive)
```

```
Consulta ativa: False
```

12. Considerações do RA2

Responda brevemente:

- O que foi possível observar com o processamento em streaming?
- Qual foi a contribuição das janelas temporais para a análise?
- Essa abordagem faria sentido em uma aplicação real do projeto? Por quê?

Na última resposta, o grupo pode reconhecer que o streaming foi utilizado apenas como exercício didático, se esse for o caso.

Resposta do grupo:

- **O que foi possível observar:** o Spark Structured Streaming trata a pasta de entrada como uma tabela sem limite e atualiza as médias a cada lote (micro-batch). Só a janela em curso muda; as já completas ficam estáveis.
- **Contribuição das janelas temporais:** agrupar por intervalos de 5 minutos transforma leituras soltas em um indicador comparável no tempo (média por janela), e deixa claro quando uma janela está completa ou ainda parcial. A tumbling foi suficiente porque cada leitura pertence a um único bloco; uma sliding suavizaria a curva, ao custo de repetir leituras entre janelas.

- **Faria sentido numa aplicação real?** Sim, em parte. Monitorar jogadores simultâneos de poucos jogos quase em tempo real (alertas de queda ou pico, acompanhamento de lançamento) combina com streaming, alimentado por um coletor da Steam Web API ou do SteamSpy no lugar da simulação. Para a análise histórica do catálogo (P1 a P3), o batch continua mais adequado. Neste trabalho o streaming foi um **exercício didático**: os dados são simulados a partir do `Peak CCU` real, não leituras reais.

RA3 — Analysis, Visualization and Machine Learning

Nesta etapa o projeto é concluído: estatística descritiva, visualizações, análise de representatividade e uma aplicação simples de aprendizagem supervisionada, sempre sobre a base preparada no RA1 (`df_limpo`). O universo analisado é o dos jogos com telemetria de audiência (`Peak CCU` > 0), porque nos demais o pico é ausência de dado, não zero real (ver RA1).

13. Análise estatística descritiva

Calcule e interprete medidas de **tendência central** (média e mediana), **dispersão** (amplitude, desvio-padrão, quartis) e **distribuição** (assimetria, percentis, concentração) das variáveis numéricas principais do projeto.

In [16]:
```
import matplotlib.pyplot as plt
import numpy as np

# Universo do RA3: jogos com telemetria de Peak CCU (> 0), como definido no
RA1
com_ccu = df_limpo[df_limpo["Peak CCU"] > 0].copy()
variaveis = ["Price", "Peak CCU", "Metacritic score", "owners_avg"]
for coluna in variaveis:  # dtypes anuláveis do pandas 3 (pd.NA) não
suportam skew(): converte para float com NaN
    com_ccu[coluna] = com_ccu[coluna].to_numpy(dtype="float64",
na_value=np.nan)

desc = com_ccu[variaveis].agg(["count", "mean", "median", "std", "min",
"max"]).T
desc["amplitude"] = desc["max"] - desc["min"]
desc["q1"] = com_ccu[variaveis].quantile(0.25)
desc["q3"] = com_ccu[variaveis].quantile(0.75)
desc["p95"] = com_ccu[variaveis].quantile(0.95)
desc["assimetria"] = com_ccu[variaveis].skew()
display(desc.round(2))

# Média vs mediana e concentração: quanto a cauda longa distorce a "média
```

```
típica"
razao = (desc["mean"] / desc["median"]).rename("média / mediana")
display(razao.round(1).to_frame())
top1 = com_ccu["Peak CCU"].nlargest(int(len(com_ccu) * 0.01)).sum() /
com_ccu["Peak CCU"].sum()
print(f"{len(com_ccu):,} jogos com Peak CCU > 0; o 1% mais jogado concentra
{top1:.1%} da soma de Peak CCU.")
print(f"Jogos gratuitos no universo: {(com_ccu['Price'] == 0).mean():.1%}.
Metacritic disponível em {com_ccu['Metacritic score'].notna().mean():.1%}.")
```

	count	mean	median	std	min	max	ampli
Price	19680.0	6.86	4.79	8.85	0.0	2.699900e+02	2.699900
Peak CCU	19680.0	340.14	3.00	9303.85	1.0	1.013936e+06	1.013935
Metacritic score	3004.0	75.94	77.00	9.20	6.0	9.700000e+01	9.100000
owners_avg	19680.0	444797.51	35000.00	3192469.13	10000.0	1.500000e+08	1.499900

	média / mediana
Price	1.4
Peak CCU	113.4
Metacritic score	1.0
owners_avg	12.7

```
19,680 jogos com Peak CCU > 0; o 1% mais jogado concentra 82.1% da soma de
Peak CCU.
Jogos gratuitos no universo: 11.1%. Metacritic disponível em 15.3%.
```

Resposta do grupo:

- **Tendência central:** a média está muito acima da mediana nas variáveis de audiência. `Peak CCU`: média 340 contra mediana 3 (razão 113 vezes); `owners_avg`: média 444.798 contra mediana 35.000 (12,7 vezes). Nelas a média descreve mal o jogo típico; a **mediana** é a medida representativa. Em `Price` (média US$ 6,86; mediana US$ 4,79) e `Metacritic score` (média 75,9; mediana 77) as duas medidas ficam próximas.
- **Dispersão:** o desvio-padrão de `Peak CCU` (9.304) é 27 vezes a média, e a amplitude vai de 1 a 1.013.936 jogadores. Entre Q1 e Q3 o intervalo é só de 1 a 15, e o percentil 95 é 328: o miolo da base é estreito e a variabilidade vem de poucos jogos extremos. Metacritic é o oposto, com dispersão pequena (desvio 9,2; Q1 a Q3 de 71 a 82).
- **Distribuição:** `Peak CCU` (assimetria 84), `owners_avg` (29) e `Price` (6,7) são fortemente assimétricas à direita; Metacritic é levemente assimétrica à esquerda (-1,15), concentrada em notas altas. O 1% mais jogado concentra **82,1%** de todo o

`Peak CCU` somado: a audiência da Steam é um mercado de poucos vencedores e uma cauda longa.

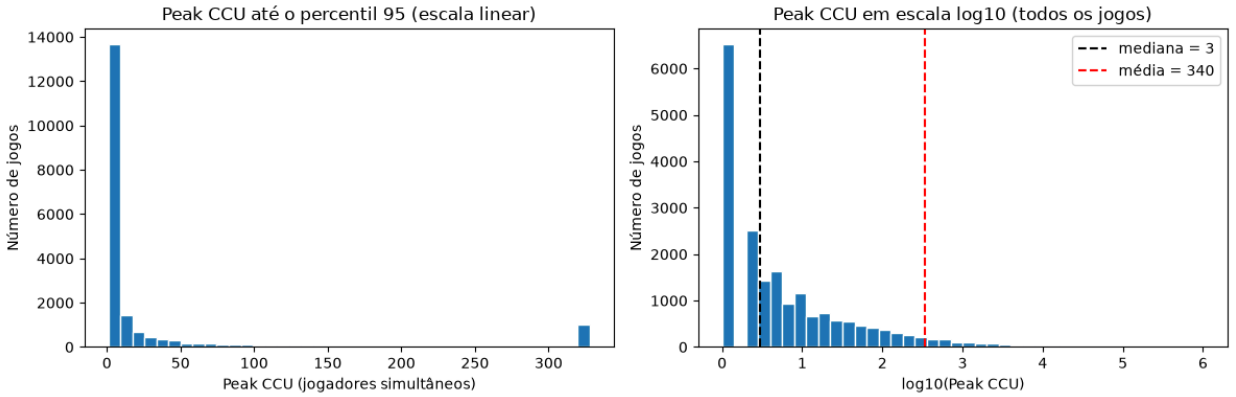
- **Consequência para o resto do RA3:** as análises e o modelo usam escala log em `Peak CCU` e `owners_avg`, e a correlação de Spearman (por postos) no lugar da de Pearson.

14. Visualização e interpretação dos dados

Quatro visualizações, cada uma com variáveis identificadas nos eixos e uma interpretação logo abaixo.

14.1 Distribuição de `Peak CCU` (histograma)

In [17]:
```
fig, eixos = plt.subplots(1, 2, figsize=(12, 4))
eixos[0].hist(com_ccu["Peak CCU"].clip(upper=com_ccu["Peak
CCU"].quantile(0.95)), bins=40, edgecolor="white")
eixos[0].set(title="Peak CCU até o percentil 95 (escala linear)",
xlabel="Peak CCU (jogadores simultâneos)", ylabel="Número de jogos")
eixos[1].hist(np.log10(com_ccu["Peak CCU"]), bins=40, edgecolor="white")
eixos[1].set(title="Peak CCU em escala log10 (todos os jogos)",
xlabel="log10(Peak CCU)", ylabel="Número de jogos")
for linha, rotulo in [(com_ccu["Peak CCU"].median(), "mediana"),
(com_ccu["Peak CCU"].mean(), "média")]:
    eixos[1].axvline(np.log10(linha), color="red" if rotulo == "média" else
"black", linestyle="--", label=f"{rotulo} = {linha:,.0f}")
eixos[1].legend()
plt.tight_layout()
plt.show()
```



Resposta do grupo:

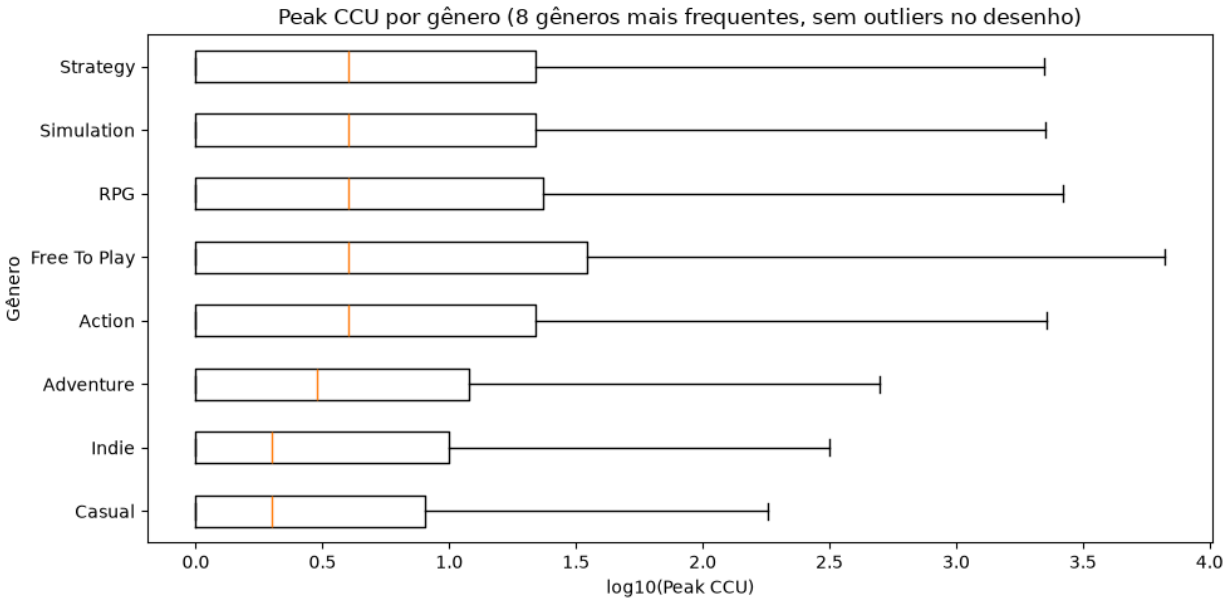
Na escala linear (painel esquerdo, já cortada no percentil 95) quase todos os jogos se acumulam na primeira barra: a maioria tem poucos jogadores simultâneos. Na escala log10 (painel direito) a forma fica legível: a massa está entre 1 e 100 jogadores (log10 de 0 a 2), e a cauda se estende até 6 (um milhão). A linha tracejada da média (340) fica bem à direita da mediana (3), o que mostra graficamente a assimetria calculada na seção 13.

Leitura: o jogo típico da Steam tem audiência mínima; "sucesso" em jogadores simultâneos é um evento raro e extremo.

14.2 `Peak CCU` por gênero (boxplot)

In [18]:
```
por_genero =
com_ccu.assign(Genre=com_ccu["Genres"].str.split(",")).explode("Genre")
por_genero["Genre"] = por_genero["Genre"].str.strip()
por_genero = por_genero[por_genero["Genre"] != "Not specified"]
top_generos = por_genero["Genre"].value_counts().head(8).index
ordem = por_genero[por_genero["Genre"].isin(top_generos)].groupby("Genre")
["Peak CCU"].median().sort_values().index

fig, eixo = plt.subplots(figsize=(10, 5))
eixo.boxplot([np.log10(por_genero.loc[por_genero["Genre"] == g, "Peak CCU"])
for g in ordem],
             tick_labels=ordem, orientation="horizontal", showfliers=False)
eixo.set(title="Peak CCU por gênero (8 gêneros mais frequentes, sem outliers
no desenho)",
         xlabel="log10(Peak CCU)", ylabel="Gênero")
plt.tight_layout()
plt.show()
display(por_genero[por_genero["Genre"].isin(top_generos)].groupby("Genre")
["Peak CCU"].agg(["count", "median", "mean"]).sort_values("median",
ascending=False).round(1))
```



Genre	count	median	mean
Action	7212	4.0	648.6
Free To Play	1557	4.0	1963.2
RPG	4931	4.0	339.3
Simulation	5385	4.0	266.1
Strategy	5045	4.0	314.2
Adventure	7887	3.0	255.6
Casual	6673	2.0	114.8
Indie	11819	2.0	147.9

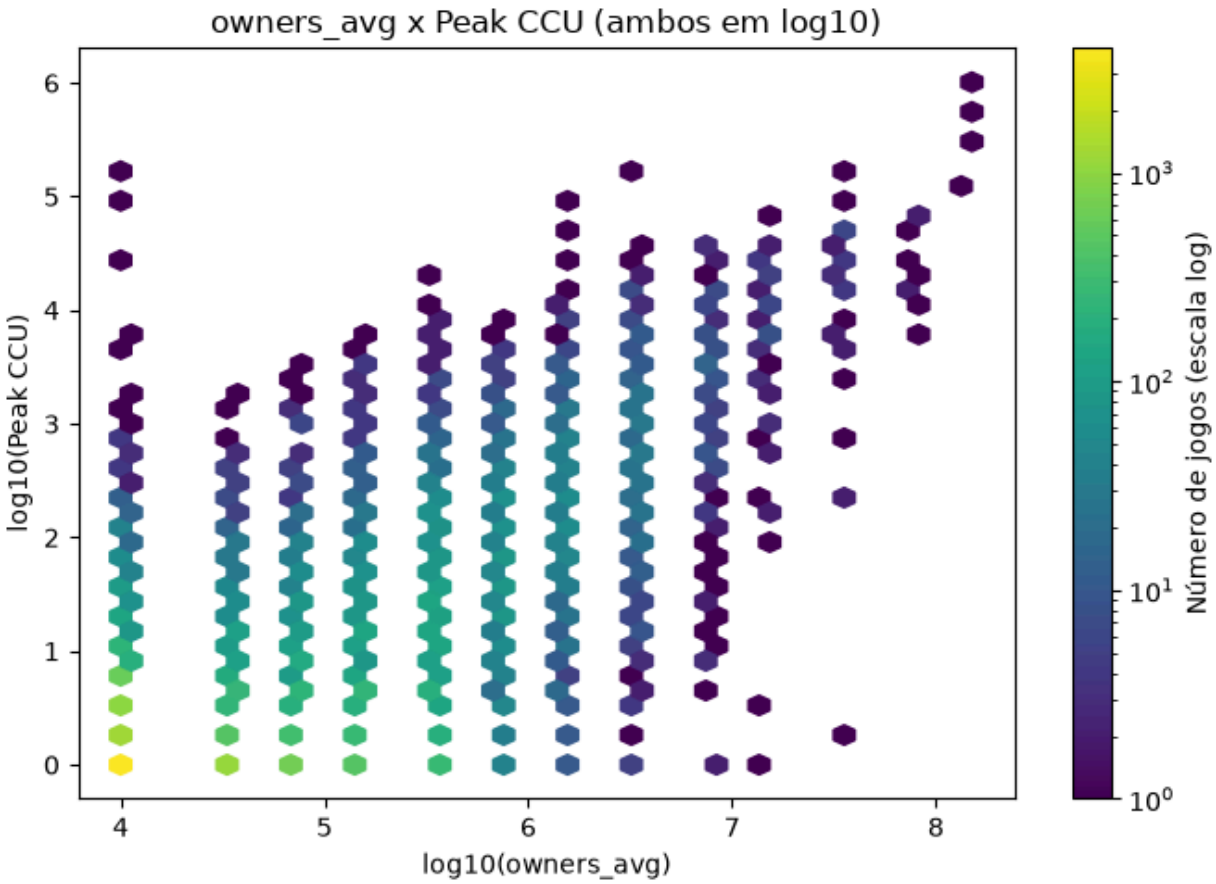
Resposta do grupo:

Os oito gêneros mais frequentes se sobrepõem muito: as medianas ficam entre 2 e 4 jogadores, e as caixas (Q1 a Q3) cobrem faixas parecidas. **Indie** e **Casual** têm a menor mediana (2), enquanto **Action**, **RPG**, **Strategy**, **Simulation** e **Free To Play** chegam a 4. A diferença entre gêneros aparece mais na **média** do que na mediana: Free To Play (1.963) e Action (649) são puxados por poucos títulos gigantes (Counter-Strike, Dota 2, PUBG), ao passo que Indie (148) e Casual (115) quase não têm esse efeito. **Leitura:** o gênero sozinho separa pouco o jogo típico; o que ele muda é a chance de existir um campeão de audiência (principalmente multiplayer e gratuito). Como um jogo tem vários gêneros, a soma das contagens passa do total de jogos.

14.3 Popularidade da base x pico de jogadores (dispersão)

In [19]:
```
from scipy.stats import spearmanr

fig, eixo = plt.subplots(figsize=(7, 5))
grade = eixo.hexbin(np.log10(com_ccu["owners_avg"]), np.log10(com_ccu["Peak
CCU"]), gridsize=40, bins="log", cmap="viridis", mincnt=1)
fig.colorbar(grade, label="Número de jogos (escala log)")
eixo.set(title="owners_avg x Peak CCU (ambos em log10)",
xlabel="log10(owners_avg)", ylabel="log10(Peak CCU)")
plt.tight_layout()
plt.show()
rho, p = spearmanr(com_ccu["owners_avg"], com_ccu["Peak CCU"])
print(f"Spearman owners_avg x Peak CCU: {rho:.2f} (p = {p:.1e})")
```




```
Spearman owners_avg x Peak CCU: 0.54 (p = 0.0e+00)
```

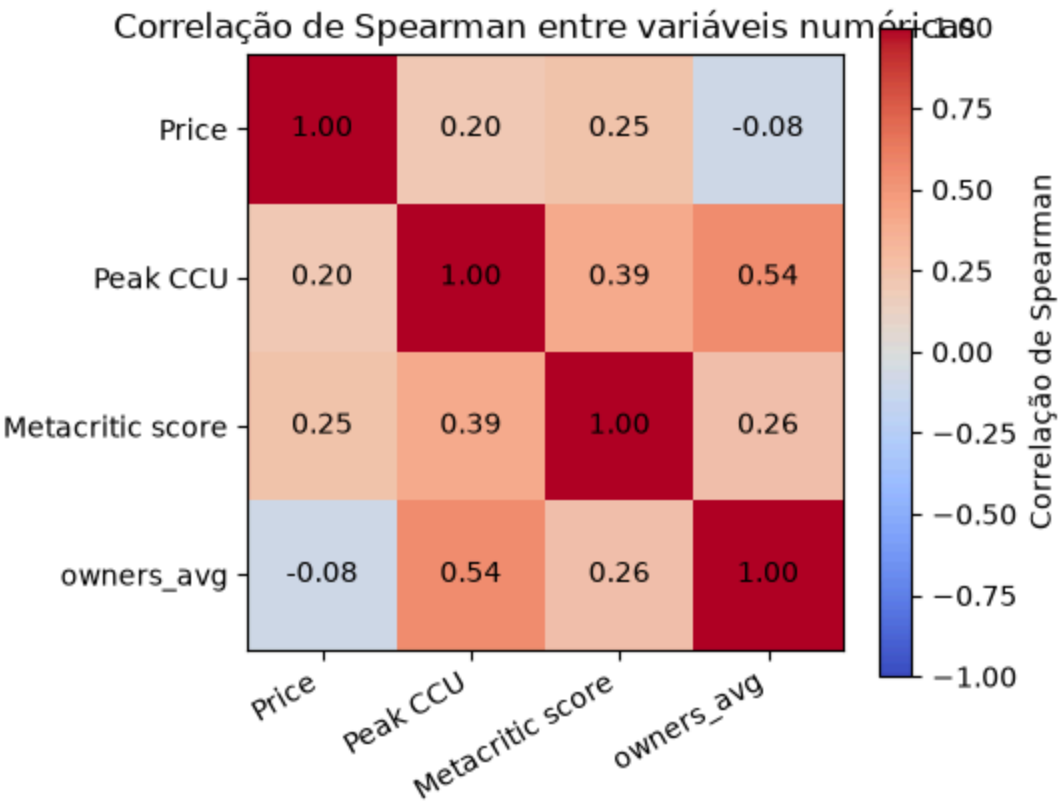
Resposta do grupo:

Há uma relação positiva clara: quanto maior a base de donos estimada, maior o pico de jogadores simultâneos (Spearman 0,54, p ≈ 0). A nuvem é larga: entre os jogos com 1,5 milhão de donos estimados, por exemplo, o `Peak CCU` vai de 4 (percentil 5) a 2.549 (percentil 95), quase três ordens de grandeza, então posse é condição necessária, mas longe de suficiente. A concentração mais densa (cor mais clara) fica na faixa de 10 mil donos com 1 a 2 jogadores simultâneos (7.779 jogos, 40% do universo). **Leitura:** popularidade acumulada e audiência simultânea andam juntas, mas não são a mesma coisa; isso reaparece no modelo da seção 16 (risco de usar `owners_avg` como feature).

14.4 Correlação entre as variáveis (mapa de calor)

In [20]:
```python
corr = com_ccu[variaveis].corr(method="spearman")
fig, eixo = plt.subplots(figsize=(5.5, 4.5))
imagem = eixo.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
eixo.set_xticks(range(len(variaveis)), variaveis, rotation=30, ha="right")
eixo.set_yticks(range(len(variaveis)), variaveis)
for i in range(len(variaveis)):
    for j in range(len(variaveis)):
        eixo.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center")
fig.colorbar(imagem, label="Correlação de Spearman")
eixo.set_title("Correlação de Spearman entre variáveis numéricas")
```

```
plt.tight_layout()
plt.show()
```



Resposta do grupo:

`Peak CCU` correlaciona mais com `owners_avg` (0,54), depois com `Metacritic score` (0,39) e, bem menos, com `Price` (0,20). O preço quase não se relaciona com a base de donos (-0,08): jogos caros não vendem mais nem menos por isso, mas têm pico um pouco maior. Metacritic se correlaciona fracamente com preço (0,25) e com donos (0,26). Como a nota Metacritic só existe para 15,3% do universo, essa correlação vale para os jogos com crítica especializada (3.004), tipicamente os maiores (viés discutido na seção 15). **Leitura:** nenhuma variável isolada explica o pico; o conjunto delas é o ponto de partida do modelo de ML.

15. Amostragem e representatividade

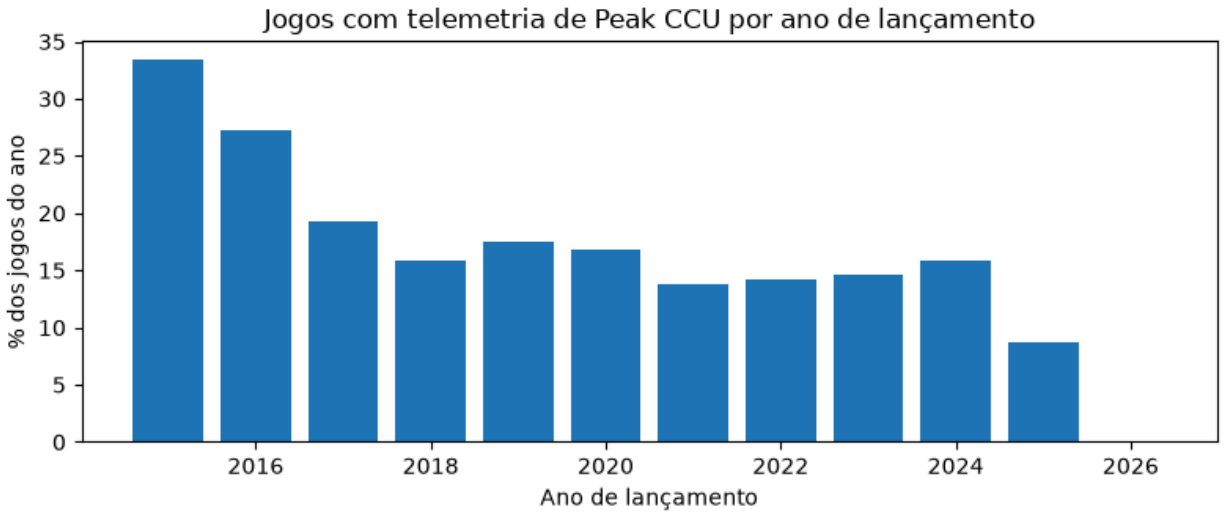
Avalie se os dados representam o problema investigado: origem e abrangência, adequação dos registros e possíveis vieses.

In [21]:
```
# O universo do RA3 (Peak CCU > 0) é um subconjunto do catálogo: compara com
o catálogo inteiro
df_limpo["tem_telemetria"] = df_limpo["Peak CCU"] > 0
comparacao_universo = df_limpo.groupby("tem_telemetria")
["Price"].agg(["count", "mean", "median"]).round(2)
comparacao_universo.index = ["sem telemetria (Peak CCU = 0)", "com
```

```
telemetria (Peak CCU > 0)"]
display(comparacao_universo)

# Proporção com telemetria por ano de lançamento
ano = df_limpo["Release date"].dt.year
por_ano = df_limpo[ano.between(2015, 2026)].groupby(ano[ano.between(2015,
2026)])["tem_telemetria"].mean()
fig, eixo = plt.subplots(figsize=(8, 3.5))
eixo.bar(por_ano.index.astype(int), por_ano.values * 100)
eixo.set(title="Jogos com telemetria de Peak CCU por ano de lançamento",
xlabel="Ano de lançamento", ylabel="% dos jogos do ano")
plt.tight_layout()
plt.show()
print(f"Catálogo: {len(df_limpo):,} jogos; universo do RA3:
{df_limpo['tem_telemetria'].sum():,}
({df_limpo['tem_telemetria'].mean():.1%}).")
```

	count	mean	median
sem telemetria (Peak CCU = 0)	106175	4.43	1.99
com telemetria (Peak CCU > 0)	19680	6.86	4.79




```
Catálogo: 125,855 jogos; universo do RA3: 19,680 (15.6%).
```

Resposta do grupo:

- **Origem e abrangência:** os dados vêm do dataset fronkongames/steam-games-dataset (Kaggle, coleta da loja Steam), com 125.855 jogos. É quase um censo do catálogo da Steam até o momento da coleta, não uma amostra sorteada; por isso não foi aplicado nenhum procedimento de amostragem. O universo analisado no RA3 é o subconjunto com telemetria (`Peak CCU` > 0): 19.680 jogos (15,6%).
- **Os registros representam o problema?** Para a pergunta "o que acompanha maior pico de jogadores", sim, mas só para jogos com audiência medida. Esse subconjunto não é aleatório: tem preço mediano de US$ 4,79 contra US$ 1,99 dos jogos sem telemetria, ou seja, tende a ser de jogos mais caros e mais estabelecidos. Metacritic existe em só 15,3% do universo.

- **Vieses e limitações:** (1) **viés de sobrevivência/telemetria**: jogos que nunca atingiram audiência mínima ficam fora, então as conclusões valem para jogos que "apareceram"; (2) **viés temporal**: a proporção com telemetria cai de 33% (2015) para 9% (2025) e 0% (2026), porque jogos recentes ainda não acumularam histórico e porque a base reflete o momento da coleta; (3) `Peak CCU` é um pico histórico único, não uma série no tempo, e `owners_avg` é ponto médio de faixa; (4) o streaming do RA2 usa leituras simuladas, não dados reais. Nada disso invalida a análise, mas os resultados **não devem ser generalizados** para todo o catálogo nem usados como causalidade.

16. Aplicação de Machine Learning

Implemente uma aplicação simples de **aprendizagem supervisionada**: defina o problema preditivo, o alvo (*target*) e as características (*features*), separe treino e teste, treine modelos, faça previsões e avalie com métricas adequadas.

16.1 Problema preditivo

Regressão: estimar o pico de jogadores simultâneos de um jogo (`Peak CCU`) a partir de características da loja. **Alvo:** `log10(Peak CCU)` (a escala log é necessária porque a variável é extremamente assimétrica, seção 13; os erros ficam em ordens de grandeza). **Universo:** os 19.680 jogos com telemetria. **Features:** preço, nota Metacritic (mediana do treino quando ausente, mais um indicador de presença), proporção de reviews positivas, log do número de reviews, ano de lançamento, oito gêneros principais (indicadores 0/1) e, no cenário completo, log de `owners_avg` .

A pergunta central é **quanto cada grupo de variáveis ajuda**. Como `owners_avg` e o número de reviews medem popularidade acumulada, quase "contêm" a resposta, então comparamos três cenários: (A) todas as features, (B) sem `owners_avg` , (C) só características intrínsecas do jogo (preço, nota, ano, gênero), sem nenhuma medida de popularidade.

In [22]:
```
import pandas as pd
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import train_test_split

alvo = np.log10(com_ccu["Peak CCU"].astype("float64"))
generos_principais =
list(com_ccu["Genres"].str.split(",").explode().str.strip().drop(labels=[],
errors="ignore")
                          .value_counts().drop("Not specified",
```

```
errors="ignore").head(8).index)

# Separação treino/teste ANTES de qualquer imputação: a mediana de
Metacritic vem só do treino
indices_treino, indices_teste = train_test_split(com_ccu.index,
test_size=0.2, random_state=42)
mediana_meta = com_ccu.loc[indices_treino, "Metacritic score"].median()
ano_lancamento = com_ccu["Release date"].dt.year.astype("float64")

X = pd.DataFrame(index=com_ccu.index)
X["Price"] = com_ccu["Price"].astype("float64")
X["metacritic"] = com_ccu["Metacritic score"].fillna(mediana_meta)
X["tem_metacritic"] = com_ccu["Metacritic score"].notna().astype(int)
avaliacoes = com_ccu["Positive"].astype("float64") +
com_ccu["Negative"].astype("float64")
X["prop_positivas"] = (com_ccu["Positive"].astype("float64") /
avaliacoes).fillna(0.5)
X["log_avaliacoes"] = np.log10(avaliacoes + 1)
X["ano"] =
ano_lancamento.fillna(ano_lancamento.loc[indices_treino].median())
for g in generos_principais:
    X["genero_" + g] = com_ccu["Genres"].str.contains(g,
regex=False).astype(int)
X["log_owners"] = np.log10(com_ccu["owners_avg"].astype("float64"))

cenarios = {
    "A) todas as features": list(X.columns),
    "B) sem owners_avg": [c for c in X.columns if c != "log_owners"],
    "C) só intrínsecas": [c for c in X.columns if c not in ("log_owners",
"log_avaliacoes", "prop_positivas")],
}
modelos = {
    "baseline (média)": lambda: DummyRegressor(strategy="mean"),
    "regressão linear": lambda: LinearRegression(),
    "random forest": lambda: RandomForestRegressor(n_estimators=200,
max_depth=12, n_jobs=-1, random_state=42),
}
print(f"Treino: {len(indices_treino):,} jogos | Teste:
{len(indices_teste):,} jogos | Features no cenário A: {len(X.columns)}")
```

```
Treino: 15,744 jogos | Teste: 3,936 jogos | Features no cenário A: 15
```

16.2 Treinamento, previsão e avaliação

Cada modelo é treinado com 80% dos jogos e avaliado nos 20% que ele nunca viu. Métricas, todas em unidades de log10(`Peak CCU`): **MAE** (erro absoluto médio), **RMSE** (penaliza erros grandes) e **R²** (fração da variância explicada; 0 equivale a prever sempre a média).

In [23]:
```
linhas, previsoes = [], {}
for nome_cenario, colunas in cenarios.items():
    for nome_modelo, criar in modelos.items():
        modelo = criar().fit(X.loc[indices_treino, colunas],
alvo.loc[indices_treino])
```

```
        previsto = modelo.predict(X.loc[indices_teste, colunas])
        real = alvo.loc[indices_teste]
        previsoes[(nome_cenario, nome_modelo)] = previsto
        linhas.append({
            "cenário": nome_cenario, "modelo": nome_modelo,
            "MAE": mean_absolute_error(real, previsto),
            "RMSE": mean_squared_error(real, previsto) ** 0.5,
            "R²": r2_score(real, previsto),
        })
resultados = pd.DataFrame(linhas).set_index(["cenário", "modelo"])
display(resultados.round(3))
melhor = resultados.loc["A) todas as features", "random forest"]
print(f"Cenário A, random forest: erro médio de {melhor['MAE']:.2f} em
log10, ou seja, fator de {10 ** melhor['MAE']:.1f}x no Peak CCU.")
```

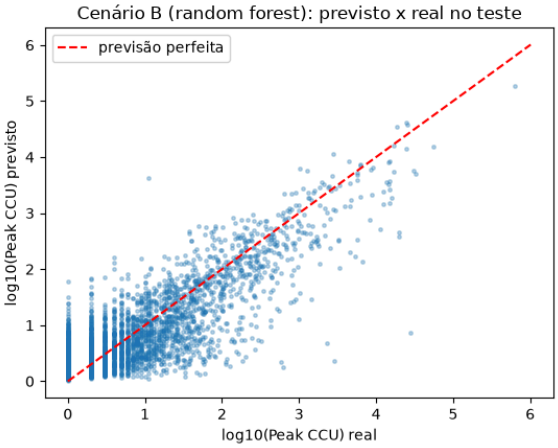
cenário	modelo	MAE	RMSE	R²
A) todas as features	**baseline (média)**	0.679	0.869	-0.000
	regressão linear	0.437	0.575	0.562
	random forest	0.346	0.465	0.713
B) sem owners_avg	**baseline (média)**	0.679	0.869	-0.000
	regressão linear	0.475	0.618	0.495
	random forest	0.346	0.467	0.711
C) só intrínsecas	**baseline (média)**	0.679	0.869	-0.000
	regressão linear	0.597	0.781	0.193
	random forest	0.574	0.761	0.234

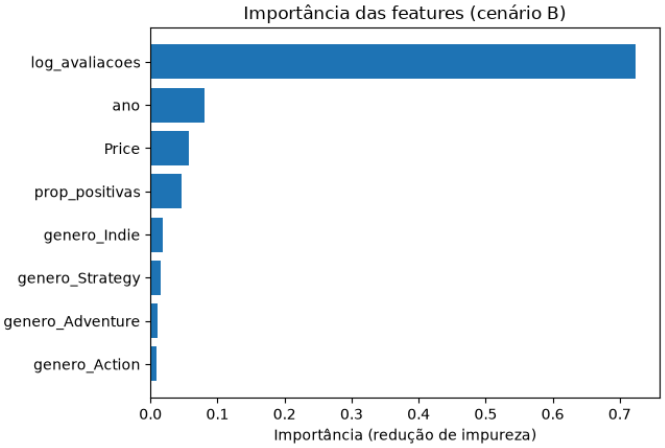
```
Cenário A, random forest: erro médio de 0.35 em log10, ou seja, fator de 2.2x
no Peak CCU.
```

In [24]:
```
# Previsto x real no teste (cenário B, random forest) e importância das
features
colunas_b = cenarios["B) sem owners_avg"]
floresta = modelos["random forest"]().fit(X.loc[indices_treino, colunas_b],
alvo.loc[indices_treino])
previsto_b = floresta.predict(X.loc[indices_teste, colunas_b])

fig, eixos = plt.subplots(1, 2, figsize=(12, 4.5))
eixos[0].scatter(alvo.loc[indices_teste], previsto_b, s=6, alpha=0.3)
limites = [alvo.min(), alvo.max()]
eixos[0].plot(limites, limites, color="red", linestyle="--", label="previsão
perfeita")
eixos[0].set(title="Cenário B (random forest): previsto x real no teste",
xlabel="log10(Peak CCU) real", ylabel="log10(Peak CCU) previsto")
eixos[0].legend()
importancia = pd.Series(floresta.feature_importances_,
index=colunas_b).sort_values().tail(8)
eixos[1].barh(importancia.index, importancia.values)
```

```
eixos[1].set(title="Importância das features (cenário B)",
xlabel="Importância (redução de impureza)")
plt.tight_layout()
plt.show()
print(importancia.sort_values(ascending=False).round(3).to_string())
```






```
log_avaliacoes      0.723
ano                 0.080
Price               0.057
prop_positivas      0.047
genero_Indie        0.019
genero_Strategy     0.016
genero_Adventure    0.010
genero_Action       0.010
```

Resposta do grupo:

- **Desempenho:** o baseline (prever sempre a média) erra 0,68 em log10 (R^2 0). O **random forest** do cenário A chega a MAE 0,35 e R^2 0,71: erra por um fator de cerca de 2,2x no `Peak CCU`, contra 4,8x do baseline. A **regressão linear** fica atrás (R^2 0,56), porque a relação com a popularidade é não linear e há interações (ano, preço, gêneros).
- **Com e sem `owners_avg`:** para a floresta, tirar `owners_avg` quase não muda nada (R^2 0,713 para 0,711), porque `log_avaliacoes` (número de reviews, outra medida de popularidade acumulada) carrega a mesma informação e responde por 72% da importância. Já na regressão linear o efeito aparece (R^2 0,56 para 0,50). Ou seja, a alta capacidade do modelo vem de **popularidade acumulada**, que é muito próxima do alvo: usar posse e reviews para prever pico de jogadores é um vazamento parcial de informação, não uma previsão "de lançamento".
- **Só características intrínsecas (cenário C):** sem nenhuma medida de popularidade, o melhor modelo explica só 23% da variância (R^2 0,23, MAE 0,57). Esse é o número honesto para a pergunta "o que o jogo em si, antes de lançar, diz sobre sua audiência": quase nada. A diferença de 0,71 para 0,23 mede quanto do resultado é popularidade.
- **Limitações:** um único split 80/20 (sem validação cruzada), hiperparâmetros fixos, apenas jogos com telemetria (viés da seção 15) e Metacritic imputado pela mediana

do treino. Os resultados valem para esse universo, não para o catálogo inteiro.

17. Conclusões

Resuma os resultados estatísticos, o desempenho do modelo, as limitações e relacione tudo à pergunta de pesquisa do projeto.

Resposta do grupo:

- **Resultados estatísticos:** a audiência na Steam é extremamente concentrada. Entre os 19.680 jogos com telemetria, o `Peak CCU` mediano é 3 e o médio 340; poucos títulos concentram quase toda a audiência (seção 13). As distribuições pedem escala log e medidas robustas (mediana, Spearman). O gênero separa pouco o jogo típico, e preço e nota têm relação fraca com o pico; a base de donos estimada é a variável mais associada (Spearman 0,54), sem ser suficiente para explicar o pico por si só (seção 14).
- **Desempenho do modelo:** random forest com R^2 0,71 e erro típico de 2,2x no `Peak CCU` quando há medidas de popularidade; R^2 0,23 quando só restam características intrínsecas do jogo. O modelo é útil para **estimar** a ordem de grandeza de jogos já conhecidos, não para prever o sucesso de um lançamento (seção 16).
- **Limitações:** só 15,6% do catálogo tem telemetria e a nota Metacritic cobre 15,3% desse universo (viés de sobrevivência e de visibilidade); `owners_avg` é uma estimativa por faixas; um único split e hiperparâmetros fixos; os dados são uma fotografia, não uma série temporal (o fluxo simulado do RA2 serve para demonstrar o processamento, não para estudar a dinâmica real).
- **Volta à pergunta de pesquisa:** quais características (preço, gênero, nota) se associam a maior pico de jogadores? **P1:** preço tem associação fraca (Spearman 0,20). **P2:** o gênero separa pouco a mediana; muda a chance de haver um campeão de audiência. **P3:** Metacritic tem associação moderada (0,39), mas só para os jogos com crítica. Juntas, as características intrínsecas explicam cerca de 23% da variância do `log10(Peak CCU)`; o restante vem de popularidade acumulada (posse, reviews) ou de fatores que a loja não descreve. **P4:** o fluxo simulado do RA2 mostra o processamento em tempo real, mas o comportamento ao vivo de verdade não foi coletado; fica como limitação.
- **Próximos passos:** validação cruzada e busca de hiperparâmetros, features de texto (descrição, tags), e dados temporais reais de `Peak CCU` para estudar lançamento e retenção.